

MySQL整理

一、整理

1、一张表，业务中会去查询这张表，where条件中有很多条件，你会怎么设计这张表的索引呢？（乐歌面试题）

首先，如果数据量比较小，建议不加索引。

其次，如果查询条件很多，我们不能把所有的查询条件都放在一个索引中，一个索引中字段的数量最好在6个左右，否则会在维护上增加很多成本，包括空间（存储）、时间（增删改的维护）。

然后对于一些选择性比较低，即重复数据比较多的字段，我们也不应该将其作为索引字段。

避免对一些字段大小比较大的字段设置为索引字段，避免造成很多的磁盘IO（如果是varchar作为索引字段，可以考虑使用前缀索引）

尽可能保证覆盖查询，减少回表的次数。

尽可能将查询比较多的条件字段、涉及到排序、分组的字段设置为索引字段。

最重要：**定期维护索引**，根据数据的变化，定期分析索引的使用情况，删除不必要的索引。

2、关于索引下推

参考文章：

- https://blog.csdn.net/weixin_74261199/article/details/146050706
- <https://cloud.tencent.com/developer/article/2334613>
- <https://baijiahao.baidu.com/s?id=1716515482593299829&wfr=spider&for=pc>
- <https://blog.csdn.net/crazymakercircle/article/details/130492417>

索引下推是MySQL5.6出现的优化手段，并默认开启。一般在联合索引中出现，对于失效的联合索引字段，在存储引擎层仍会进行条件筛选，explain执行计划Extra的值为Using index condition。带着联合索引的全部字段去存储引擎查找数据，缩小数据量，可以有效减少回表的次数，大大提升了查询的效率。如果没有索引下推，失效的联合索引字段并不会在存储引擎层筛选，而是返回到server层进行二次筛选。

概括：当对联合索引进行范围查询的时候，会导致后面的索引列失效，从而需要回表数据交给server层进行条件判断，ICP解决的问题是 在引擎层直接对未生效的索引列进行条件判断，从而减少回表次数提高效率。

下推：其实就是指将部分上层（服务层）负责的事情，交给了下层（引擎层）去处理。

启停用：

```
# 查看索引下推是否开启
select @@optimizer_switch
# 开启索引下推
set optimizer_switch="index_condition_pushdown=on"
# 关闭索引下推
set optimizer_switch="index_condition_pushdown=off"
```

示例：参考文章中

问题：

1、为什么后面的level会失效呢

```
select * from user where name like "大%" and level = 1;
```

索引是 (name, level)，在实际执行的时候后面的level会失效，但还是会走 (name, level) 索引因为这里涉及到了最左前缀原则和范围查询，因此范围查询后的字段就会失效。

组合索引可以抽象的把索引当成 '张三19', '张三20', '张四19', '李五12', (后面的数字代表年龄), 他们都是按照自然顺序排列. 如果 like '张%' and age = 19 的话, 先like '张%' 匹配到 '张三19', '张三20', '张四19', 但是年龄的顺序是 19, 20, 19 因此无法通过B+树的索引查找, 但是 如果是 name = '张三' and age = 19, 先使用 name = '张三' 索引到 '张三19', '张三20', 此时年龄的顺序也是根据大小排好序的 19, 20, 此时 age = 19 也可以索引

2、对于回表

一个查询正常来说只能使用一个索引，回表只是为了获取完整的数据。当查询条件需要的字段不在索引中时，数据库会通过索引查找到匹配的记录（例如，索引的主键或其他字段），但因为索引只是存储了部分信息（如字段的前缀、索引列的值等），因此需要通过 **回表** 去获取实际的记录数据，以补充索引中缺失的字段内容。

3、服务层二次过滤

如果数据库查询无法通过索引进行有效的下推过滤（比如通过索引只查到了部分数据，但查询条件要求更多的信息），那么 **数据库会先执行基本的查询**，然后返回符合索引条件的部分数据。接着，在服务层或者应用层，你可以根据需要进行 **二次过滤**，进一步缩小或筛选符合条件的数据。

举个例子，假设查询条件是 `WHERE name = 'JohnDoeExtra'`，但是前缀索引只适用于前 10 个字符。数据库可能会回表获取所有 `name` 字段为 'JohnDoeExtr' 的记录，回表获取全部数据，然后在应用层或服务层会进一步筛选出符合 `name = 'JohnDoeExtra'` 的记录。

什么时候需要服务层二次过滤：

- **无法通过索引过滤时**：如果查询条件涉及的字段未被索引或索引无法覆盖所有过滤条件，数据库就需要回表，或者在返回的结果集上进行额外的筛选。
- **复杂的条件无法通过索引下推**：例如，当查询条件包含复杂的运算、函数调用或不可优化的模糊匹配（如 `%John%`），这些条件就可能无法通过索引下推去直接过滤数据，数据库只能返回更多数据后交由应用层进一步处理。

3、分页查询（深分页）

<https://cloud.tencent.com/developer/article/2356472>

如果不用回表其实也没关系，但是要回表的深分页一定很吃性能！所以要减少回表，竟可能利用索引覆盖，减少回表的次数。

二、补充

1、关于一台服务器能部署多少mysql服务实例

在大多数情况下，一个服务器上可以运行多个MySQL服务实例，但需要满足以下条件：

- **不同端口**：每个实例需使用不同的端口号。即端口不能相同
- **不同数据目录**：每个实例应有独立的数据目录，避免数据冲突。即数据库目录不能相同
- **不同配置文件**：每个实例应有独立的配置文件，指定各自的端口、数据目录等。即配置文件安装路径不能相同
- **不同套接字文件**（如适用）：每个实例应有独立的套接字文件。即socket文件的生成路径不能相同

实现方式：

- 多配置文件启动不同进程来实现多实例

修改配置文件（如 `my.cnf` 或 `my.ini`）为每个实例设置不同的端口、数据目录和套接字文件。

示例：两个实例，配置文件如下

```
[mysqld1]
port=3306
datadir=/var/lib/mysql1
socket=/var/lib/mysql1/mysql.sock

[mysqld2]
port=3307
datadir=/var/lib/mysql2
socket=/var/lib/mysql2/mysql.sock
```

使用参考文档：<http://blog.csdn.net/clevercode/article/details/47610619>

- 通过官方自带的mysqld_multi实现多实例

使用单独的配置文件来实现多实例，这种方式定制每个实例的配置不太方便，优点是管理起来很方便，集中管理。

使用参考文档：https://blog.csdn.net/gg_24700495/article/details/144392959

（注意：推荐使用多个配置文件方式。这种实际应用中好，**耦合性**不强，配置方便，特别是主从复制的时候。）

注意：

- **资源分配**：确保服务器有足够的资源支持多个实例。
- **管理复杂度**：多个实例会增加管理难度，需确保配置正确。

2、关于套接字配置

```
[mysqld1]
port=3306
datadir=/var/lib/mysql1
socket=/var/lib/mysql1/mysql.sock

[mysqld2]
port=3307
datadir=/var/lib/mysql2
socket=/var/lib/mysql2/mysql.sock
```

可以看到在mysql的配置中，有指定一个配置项：socket

- 第一个 MySQL 实例使用 `/var/lib/mysql1/mysql.sock` 作为套接字文件。
- 第二个 MySQL 实例使用 `/var/lib/mysql2/mysql.sock` 作为套接字文件。

在 MySQL 配置文件中，`socket` 参数用于指定 **MySQL 服务器使用的 Unix 套接字文件 (Unix Socket File)** 的路径。Unix 套接字是一种用于在同一台机器上进行进程间通信 (IPC) 的机制。

1.socket的作用：

- 当 MySQL 客户端和服务端运行在同一台机器上时，客户端可以通过 Unix 套接字文件与服务器通信，而不需要通过网络协议（如 TCP/IP）。
- 使用 Unix 套接字的优点是速度更快，因为不需要经过网络栈，同时也不需要占用端口。

2.客户端连接方式（使用Socket进行连接）：

- 如果客户端和 MySQL 服务器在同一台机器上，客户端可以通过指定 `socket` 参数来连接服务器。
- 使用 `mysql` 命令行客户端连接：

```
mysql -S /var/lib/mysql1/mysql.sock -u root -p
```

- `-S` 参数指定了 Unix 套接字文件的路径。
- 如果不指定 `-S`，客户端会尝试使用默认的套接字文件路径（通常是 `/tmp/mysql.sock` 或 `/var/lib/mysql/mysql.sock`）

3.关于Socket文件的默认路径：

- 如果没有显式配置 `socket` 参数，MySQL 会使用默认的套接字文件路径。
- 默认路径通常为：
 - `/tmp/mysql.sock`
 - `/var/lib/mysql/mysql.sock`

具体路径取决于操作系统和 MySQL 的安装方式

4.关于Socket与TCP/IP的区别：

- **Socket (Unix Domain Socket) :**
 - 仅适用于同一台机器上的进程间通信。
 - 性能更高，因为不需要经过网络栈。
 - 不需要占用端口。

- **TCP/IP:**
 - 适用于远程连接或本地连接。
 - 需要占用端口（如 3306）。
 - 性能略低于 Unix 套接字。

5.多实例配置Socket:

- 在同一台服务器上运行多个 MySQL 实例时，每个实例必须使用不同的套接字文件路径，以避免冲突。
- 例如：

```
[mysqld1]
socket=/var/lib/mysql1/mysql.sock

[mysqld2]
socket=/var/lib/mysql2/mysql.sock
```

这样，两个实例可以同时运行，客户端可以通过不同的套接字文件连接到不同的实例

6.注意

- 确保 `socket` 文件所在的目录有正确的权限，MySQL 服务器和客户端用户需要能够访问该文件。
- 如果删除了 `socket` 文件，MySQL 服务器需要重启才能重新生成它。
- 如果客户端和服务端不在同一台机器上，必须使用 TCP/IP 连接，而不能使用 Unix 套接字。

总结:

`socket` 参数用于指定 MySQL 服务器使用的 Unix 套接字文件路径，适用于同一台机器上的客户端与服务端通信。在多实例配置中，每个实例需要配置不同的 `socket` 路径以避免冲突。通过 Unix 套接字连接可以提高性能，但仅限于本地连接。

3、关于MySQL实例的连接数上限的决定因素

一个 MySQL 实例的连接数上限主要由以下几个因素决定：

1.MySQL系统变量 `max_connections`

这是 MySQL 实例允许的最大并发连接数。默认值通常为 151，但可以根据服务器配置和需求调整。

可以通过以下命令查看和修改：

```
SHOW VARIABLES LIKE 'max_connections'; -- 查看当前值
SET GLOBAL max_connections = 500;      -- 修改全局值（重启后失效）
```

永久修改需要在配置文件（如 `my.cnf` 或 `my.ini`）中设置：

```
[mysqld]
max_connections = 500
```

每当一个客户端（应用程序、用户或工具）请求与 MySQL 服务器建立连接时，都会占用一个 `max_connections` 计数。以下是一些可能会占用 `max_connections` 数量的情况：

- 客户端连接

每个主动请求数据库的客户端都会占用一个连接：一个 Web 应用程序的每个用户请求；一个数据库客户端工具（如 MySQL Workbench 或 phpMyAdmin）打开的每个连接；其他服务或应用（例如后台进程、分析工具、监控系统）发出的数据库连接。

- 连接池中的连接

许多应用程序使用数据库连接池来管理与数据库的连接。在连接池中，多个请求可能会共享一个或多个连接。因此，即使没有实际的请求，连接池中的空闲连接也会占用

`max_connections` 配额。

- 长时间运行的查询或未释放的连接：

如果某个查询执行时间非常长（例如复杂的查询或数据库未优化的查询），它可能会保持连接打开并占用 `max_connections`。长时间运行的查询会消耗一个连接，直到查询完成。如果应用程序没有正确关闭连接或者连接没有及时释放，可能会导致连接一直占用

`max_connections`。这种情况通常发生在数据库连接池中，或是开发者代码中未正确关闭数据库连接。有时某些查询可能会被锁定或阻塞，导致连接在等待状态下被占用。例如，在一个事务中，如果有行锁或表锁未释放，连接可能会被长时间占用。

- 复制连接

如果数据库配置了主从复制，那么主服务器与从服务器之间的连接也会占用

`max_connections`。这些连接通常是为了数据同步而保持的。

可以通过如下命令查看占用的连接：

```
SHOW PROCESSLIST;
```

这个命令会列出所有当前的连接，包括每个连接的 ID、状态、执行的查询等信息。它帮助你识别哪些连接可能正在占用资源或阻塞。

2.操作系统限制

MySQL 依赖操作系统的文件描述符（File Descriptors，文件描述符是操作系统用来管理打开文件（包括网络连接、管道、设备等）的一种机制）来处理连接。如果操作系统的文件描述符限制较低，即使 `max_connections` 设置得再高，也无法建立更多连接。可以通过以下命令查看和修改文件描述符限制：

```
ulimit -n # 查看当前 shell 会话中单个进程可以打开的文件描述符的最大数量
ulimit -n 65535 # 临时设置当前 shell 会话中单个进程可以打开的文件描述符的最大数量
```

永久修改需要在 `/etc/security/limits.conf` 中设置：

```
* soft nofile 65535
* hard nofile 65535
```

- `soft` 表示软限制（当前会话可以修改的最大值）。
- `hard` 表示硬限制（系统允许的最大值）。
- `nofile` 表示文件描述符的数量。

关于MySQL和文件描述符的关系

- MySQL 的每个客户端连接都会占用一个文件描述符（连接mysql需要先到服务器吧，服务器处理连接就需要通过文件描述符去接受）。

- 如果操作系统的文件描述符限制较低，即使 MySQL 的 `max_connections` 设置得很高，也无法建立更多连接。
- 因此，`ulimit -n` 的值会直接影响 MySQL 能够支持的最大连接数。

3.MySQL内存资源

关于占用内存的系统变量`thread_stack` 和 `max_allowed_packet`

`thread_stack` 和 `max_allowed_packet` 是 MySQL 中两个重要的系统变量，分别用于控制线程栈大小和网络数据包的最大大小。

1、thread_stack

作用：

- `thread_stack` 定义了每个线程的栈大小（单位为字节）。
- 线程栈用于存储线程执行时的局部变量、函数调用信息等。

默认值：不同平台的默认值不同，通常为 256KB（262144 字节）或 192KB（196608 字节）

查看/修改当前值：

```
SHOW VARIABLES LIKE 'thread_stack';
```

```
[mysql]  
thread_stack = 262144
```

注意：

- 如果 `thread_stack` 设置过小，可能会导致栈溢出，进而引发 MySQL 崩溃。
- 如果 `thread_stack` 设置过大，会占用更多内存，尤其是在高并发场景下，可能会影响系统性能

2、max_allowed_packet

作用：

`max_allowed_packet` 定义了 MySQL 服务器和客户端之间传输的单个数据包的最大大小（单位为字节）。

当查询大表时，如果结果集的大小超过了 `max_allowed_packet` 的限制，MySQL 服务器会尝试将结果集拆分成多个数据包发送。但如果单个数据包的大小仍然超过 `max_allowed_packet`，MySQL 会报错并中断查询。

它影响以下操作：

- 客户端发送到服务器的 SQL 语句大小。
- 服务器返回给客户端的结果集大小。
- 复制（Replication）过程中传输的数据包大小。

默认值：默认值为 4MB（4194304 字节）

注意事项：

- 如果 `max_allowed_packet` 设置过小，可能会导致以下问题：
 - 插入或更新大字段（如 BLOB、TEXT）时失败。

- 查询返回大量数据时失败。
- 如果 `max_allowed_packet` 设置过大，可能会占用更多内存，尤其是在处理大字段或大结果集时。

除了 `thread_stack` 和 `max_allowed_packet`，还有一些其他的参数会影响每个连接的内存占用：

- `sort_buffer_size`：每个连接会分配一定的内存用于排序操作。如果查询涉及排序，MySQL 会为该连接分配 `sort_buffer_size` 字节的内存。该参数默认值通常为 256KB。
- `read_buffer_size`：每个连接会分配一定的内存用于读取数据。当执行全表扫描时，MySQL 会为每个连接分配 `read_buffer_size` 字节的内存。默认值为 128KB。
- `join_buffer_size`：用于执行连接操作的内存。默认值通常为 256KB。

总体内存占用计算示例：

每个连接的内存占用总量由多个因素决定，包括上述参数以及其他的缓存、内部结构等。假设 MySQL 配置了以下默认值：

- `thread_stack = 196608` 字节 (192KB)
- `max_allowed_packet = 4194304` 字节 (4MB)
- `sort_buffer_size = 262144` 字节 (256KB)
- `read_buffer_size = 131072` 字节 (128KB)
- `join_buffer_size = 262144` 字节 (256KB)

那么每个连接的内存占用大概是这些值的总和（不考虑其他内部内存分配）：

```
192KB (thread_stack)
+ 4MB (max_allowed_packet)
+ 256KB (sort_buffer_size)
+ 128KB (read_buffer_size)
+ 256KB (join_buffer_size)
= 大约 5.8MB 每个连接
```

注意：

- 这些内存是**每个连接**独立分配的。如果有多个连接同时打开，那么每个连接都会占用这些内存资源。
- 如果服务器处理大量的并发连接，那么这些参数的设置会直接影响到 MySQL 的总内存消耗。因此，合理配置这些参数非常重要，尤其是 `max_allowed_packet` 和 `thread_stack`。

内存优化建议：

- **减少不必要的内存占用**：如果你的应用不需要处理非常大的数据包，可以将 `max_allowed_packet` 设置为一个较小的值来减少每个连接的内存占用。
- **调整 `thread_stack` 大小**：默认情况下 `thread_stack` 的大小可能是较大的。大多数应用不需要过多的栈空间，因此可以根据实际需求减少 `thread_stack`。
- **监控连接数**：如果连接数过多，而每个连接的内存开销较大，会导致服务器内存消耗过多。可以通过优化应用程序的数据库连接池管理来减少不必要的连接数量。

每个连接都会占用一定的内存资源（主要由系统变量 `thread_stack` 和 `max_allowed_packet` 决定，还有其他因素）。当前MySQL实例所在服务器如果内存不足，可能会导致无法建立新连接，可以通过以下公式估计连接数的上线：

$$\text{总可用内存} / \text{每个连接占用的内存} \approx \text{最大连接数}$$

4.MySQL线程池限制

如果启用了线程池插件（Thread Pool Plugin），连接数上限还会受到线程池配置的限制。

相关系统变量包括：

- `thread_pool_size`：线程池大小。
- `thread_pool_max_threads`：最大线程数。

MySQL 线程池功能是 **企业版**（MySQL Enterprise Edition）的一部分，社区版（MySQL Community Edition）并不提供线程池功能。您需要检查您是否使用的是企业版。

MySQL 线程池是 MySQL 服务器用于管理并发连接请求的机制。它通过复用一定数量的工作线程来提高处理并发连接的效率，避免频繁创建和销毁线程的开销。对于高并发的数据库应用，线程池能显著提升性能，特别是在大量客户端连接到 MySQL 时。

默认情况下，MySQL 为每个客户端连接分配一个单独的线程。这样当连接数较高时，MySQL 会创建大量线程来处理并发连接。然而，创建和销毁线程会消耗系统资源，尤其是在高并发的情况下，可能会导致性能瓶颈。为了优化这个问题，MySQL 提供了线程池机制来复用线程，从而减少线程创建和销毁的开销。

线程池工作原理：

- 线程池在 MySQL 服务器启动时创建一定数量的工作线程（而不是为每个连接创建独立线程）。
- 当有客户端连接请求时，线程池会从池中分配一个空闲线程来处理该请求。
- 处理完成后，线程会返回到池中等待下一个连接请求，而不是销毁。
- 线程池通过限制并发执行的线程数来避免系统过载，优化性能。

线程池优点：

- **提高并发性能**：通过线程复用来减少线程的创建与销毁开销。
- **减少资源消耗**：减少了线程调度和上下文切换的开销，尤其是在高并发环境下，系统资源（CPU、内存等）的消耗会降低。
- **改善负载均衡**：线程池可以平衡处理请求的工作负载，避免线程资源的过度竞争。

配置MySQL线程池：

1.确保支持线程池

MySQL 线程池功能是 **企业版**（MySQL Enterprise Edition）的一部分，社区版（MySQL Community Edition）并不提供线程池功能。您需要检查您是否使用的是企业版。

2.配置文件设置

线程池的设置可以通过 MySQL 配置文件（如 `my.cnf` 或 `my.ini`）中的以下参数来进行配置：

```
[mysqld]
# 启用线程池功能
thread_handling=pool-of-threads

# 线程池大小（最小工作线程数）
thread_pool_size=16

# 线程池的最大连接数
thread_pool_max_threads=100

# 线程池的最小连接数
thread_pool_min_threads=8
```

- `thread_handling=pool-of-threads`：启用线程池机制，确保 MySQL 使用线程池而不是传统的每个连接一个线程的模式。
- `thread_pool_size`：配置线程池中最小线程数。该值设置了在空闲时 MySQL 将维持的工作线程数量。
- `thread_pool_max_threads`：设置线程池中最大线程数。超过此值的连接将被排队等待。
- `thread_pool_min_threads`：设置线程池中最小工作线程数。该值决定了线程池在启动时将保持的最小线程数。

3.动态调整

您也可以在运行时通过 `SET GLOBAL` 命令调整线程池的配置：

```
-- 设置线程池最大线程数
SET GLOBAL thread_pool_max_threads = 100;

-- 设置线程池最小线程数
SET GLOBAL thread_pool_min_threads = 8;

-- 设置线程池工作线程数
SET GLOBAL thread_pool_size = 16;
```

4.查看线程池状态

使用以下命令可以查看线程池的当前状态和活动信息：

```
SHOW STATUS LIKE 'Thread_pool%';
```

这些状态信息可以帮助你分析线程池的运行情况，并进一步优化配置。

5.客户端和服务器的网络配置

如果客户端或服务器的网络配置限制了并发连接数（如防火墙规则、TCP/IP 连接数限制等），也会影响 MySQL 的连接数上限。

6.MySQL 版本和存储引擎

- 不同版本的 MySQL 对连接数的支持可能有所不同。
- 某些存储引擎（如 InnoDB）可能会占用更多资源（一个连接需要占用更多内存，导致连接数的数量减少），从而间接影响连接数上限。

4、关于MySQL的结果集

在MySQL中，执行器通过存储引擎API获取的数据，最终会放到**结果集 (Result Set)** 中。结果集的存储位置和形式取决于查询的类型、数据量以及MySQL的配置。以下是结果集存储的几种常见情况：

1.结果集存储在内存中

对于大多数查询，尤其是数据量较小的查询，MySQL会将结果集直接存储在**内存**中。

特点

- **速度快**：内存访问速度快，适合小数据量。
- **内存限制**：如果结果集过大，可能导致内存不足，进而影响性能。
- **适用场景**：小规模查询、OLTP（在线事务处理）场景。

内存中的存储结构

- MySQL使用内部数据结构（如 `THD`、`Query_result` 等）来管理结果集。
- 结果集通常以行（Row）为单位存储，每行数据以行的格式（如 `InnoDB` 的行格式）存储在内存中。

2.结果集存储在磁盘

如果结果集非常大，超出了内存的限制，MySQL会将部分结果集存储在**磁盘**上。

特点

- **避免内存耗尽**：适合处理大规模数据查询。
- **性能较低**：磁盘I/O速度较慢，可能导致查询性能下降。
- **适用场景**：大规模数据查询、OLAP（在线分析处理）场景。

磁盘存储的实现

- MySQL使用临时文件（Temporary File）来存储超大的结果集。
- 临时文件通常位于MySQL的临时目录（由 `tmpdir` 参数指定）。

3.结果集的流式处理

对于某些查询（如 `SELECT` 语句），MySQL支持**流式处理**，即边查询边返回结果，而不是一次性将结果集存储在内存或磁盘中。

特点

- **内存占用低**：不需要一次性加载所有数据。
- **响应速度快**：客户端可以尽快收到第一批数据。
- **适用场景**：大数据量查询、分页查询。

流式处理的实现

- MySQL通过存储引擎API逐行获取数据，并直接发送给客户端。
- 客户端可以逐步接收数据，而不需要等待整个查询完成。

4.结果集的缓存

如果查询结果被缓存（如使用查询缓存或应用程序缓存），结果集可能存储在缓存中。

特点

- **加速查询**：缓存命中时，直接返回结果，无需执行查询。
- **缓存失效**：数据更新时，缓存需要失效。
- **适用场景**：读多写少的场景。

缓存存储的实现

- MySQL的查询缓存（Query Cache）将结果集存储在内存中。
- 应用程序缓存（如Redis）也可以存储结果集。

5.结果集的存储位置总结

存储位置	特点	适用场景
内存	速度快，但受内存限制	小规模查询、OLTP场景
磁盘	避免内存耗尽，但性能较低	大规模查询、OLAP场景
流式处理	内存占用低，响应速度快	大数据量查询、分页查询
缓存	加速查询，但需要处理缓存失效	读多写少的场景

6.控制结果集存储位置

- **内存限制**：通过 `max_heap_table_size` 和 `tmp_table_size` 参数控制内存中临时表的大小。
- **磁盘存储**：当内存不足时，MySQL会自动将结果集写入磁盘。
- **流式处理**：通过 `mysql_use_result` 或 `mysql_store_result` API控制是否流式返回结果。
- **查询缓存**：通过 `query_cache_type` 和 `query_cache_size` 参数控制查询缓存的使用。

注意，结果集存储在内存中不是存放在 **InnoDB Buffer Pool**

InnoDB Buffer Pool 是 InnoDB 存储引擎的核心，它用于缓存 **InnoDB 表** 的 **数据页和索引页**。它确保从磁盘加载的数据能够快速访问。对于 **内存临时表**（使用 MEMORY 引擎的表），它们的内容并不会存放在 InnoDB Buffer Pool 中，而是存放在系统的内存中，并且这些表通常不是持久化的。

如果 **内存临时表** 的大小超过了配置的内存限制（例如 `tmp_table_size` 或 `max_heap_table_size`），MySQL 会将其转换为 **磁盘临时表**，通常使用 **InnoDB** 或 **MyISAM** 存储引擎存储在磁盘上，而不是在内存中

5、MySQL的EXPLAIN语句之Extra字段

https://blog.csdn.net/qg_38671485/article/details/145118810

除了 `Using filesort`，MySQL 查询执行过程中可能还会采用其他几种排序方式：

- `Using index`：如果查询使用了覆盖索引（covering index），即所有需要的字段都可以从索引中获取，MySQL 会直接使用索引中的数据进行排序，而无需额外的排序操作。这比 `Using filesort` 更高效，因为没有额外的磁盘 I/O 操作。

- **Using temporary**: 当 MySQL 查询需要创建临时表时, 可能会显示 **Using temporary**。如果查询需要排序, 且排序不能通过索引来优化, MySQL 会在执行时使用临时表。这有时和 **Using filesort** 一起出现。
- **Using index condition**: 这表明 MySQL 使用了索引条件下推 (Index Condition Pushdown), 这是一个优化器的策略, 它通过将部分查询条件下推到索引扫描阶段来提高性能。
- **Using where**: 表示 MySQL 使用了 WHERE 子句来限制返回的记录。通常这和其他操作一起出现在执行计划中。

6、关于根据索引进行数据查询相关内容汇总

1、示例

关于根据索引进行数据查询过滤、筛选的全过程示例。

表: account

索引: update_time (idx_update_time)

查询语句:

```
select id,name,balance from account where update_time> '2020-09-19' limit 100000,10;
```

具体执行过程:

1、回表的时机

(1) 索引查找

- MySQL会使用 **update_time** 索引 (**idx_update_time**) 查找满足 **update_time > '2020-09-19'** 的所有行。
- 由于 **update_time** 是索引字段, MySQL可以通过索引快速定位到满足条件的第一行, 然后顺序扫描索引, 直到找到所有满足条件的行。

(2) 回表

- 索引 **idx_update_time** 只包含 **update_time** 和主键 **id** 的值。
- 查询需要返回 **id**、**name** 和 **balance** 字段, 而 **name** 和 **balance** 不在索引中, 因此需要**回表** (即通过主键 **id** 访问聚簇索引) 获取完整的数据行。
- MySQL并不是每找到一条符合 **update_time** 条件的数据就立即回表。
- 而是**先通过索引找到满足条件的主键 id, 然后批量回表**。
- 具体来说, MySQL会先通过索引定位到满足 **update_time > '2020-09-19'** 的主键 **id**, 然后根据这些 **id** 回表获取完整的数据行。

2、LIMIT 的过滤时机

LIMIT 的过滤是在**服务层**进行的, 而不是在存储引擎层。

(1) 存储引擎层

- 存储引擎负责通过索引查找满足 **update_time > '2020-09-19'** 的主键 **id**。

- 存储引擎会将满足条件的主键 `id` 返回给服务层。

(2) 服务层

- 服务层根据主键 `id` 回表获取完整的数据行。
- 服务层会应用 `LIMIT 100000, 10`，跳过前100000行，返回接下来的10行。

3、LIMIT 的工作方式：

- MySQL需要先找到满足条件的前100010行（100000 + 10），然后跳过前100000行，返回最后10行。
- 这意味着存储引擎需要返回满足条件的前100010行给服务层，服务层再进行过滤。

4、回表次数：

- **回表次数 = 满足条件的前100010行。**
- 对于每一行，MySQL都需要通过主键 `id` 回表获取 `name` 和 `balance` 字段。

因此，回表次数是100010次。

优化建议

由于回表次数较多，查询性能可能会较差。以下是一些优化建议：

1) 延迟关联

通过子查询先获取主键 `id`，然后再回表获取其他字段。例如：

```
SELECT id, name, balance
FROM account
JOIN (
  SELECT id
  FROM account
  WHERE update_time > '2020-09-19'
  LIMIT 100000, 10
) AS tmp ON account.id = tmp.id;
```

- 子查询只返回 `id`，可以利用索引快速定位。
- 外层查询通过 `id` 回表获取 `name` 和 `balance` 字段，回表次数减少到10次。

2) 覆盖索引

如果查询只需要索引字段，可以使用覆盖索引避免回表。例如：

```
SELECT id, update_time FROM account WHERE update_time > '2020-09-19' LIMIT 100000, 10;
```

- 如果索引 `idx_update_time` 包含 `update_time` 和 `id`，MySQL可以直接从索引中获取数据，无需回表。

3) 优化分页

如果分页深度较大（如 `LIMIT 100000, 10`），可以考虑使用基于主键的分页优化。例如：

```
SELECT id, name, balance
FROM account
WHERE update_time > '2020-09-19' AND id > ?
ORDER BY id
LIMIT 10;
```

- ? 是上一页的最后一个 id 值。
- 通过主键 id 和 update_time 联合过滤，减少扫描的行数。

5. 总结

- 回表次数是100010次，因为MySQL需要找到满足条件的前100010行，然后应用 LIMIT 100000, 10。
- LIMIT 的过滤是在服务层进行的，存储引擎只负责返回满足条件的主键 id。
- 为了优化查询，可以使用延迟关联、覆盖索引或优化分页策略。

2、关于索引查找的过程

- 等值查找

```
SELECT * FROM users WHERE age = 25;
```

MySQL会使用 idx_age_city 索引，定位到 age = 25 的行。

- 范围查询

```
SELECT * FROM users WHERE age > 18 AND city = 'New York';
```

MySQL会使用 idx_age_city 索引，先定位到 age > 18 的行，然后过滤 city = 'New York'。

- 前缀匹配

```
SELECT * FROM users WHERE name LIKE 'John%';
```

如果 name 有索引，MySQL会使用索引定位到 name 以 John 开头的行。